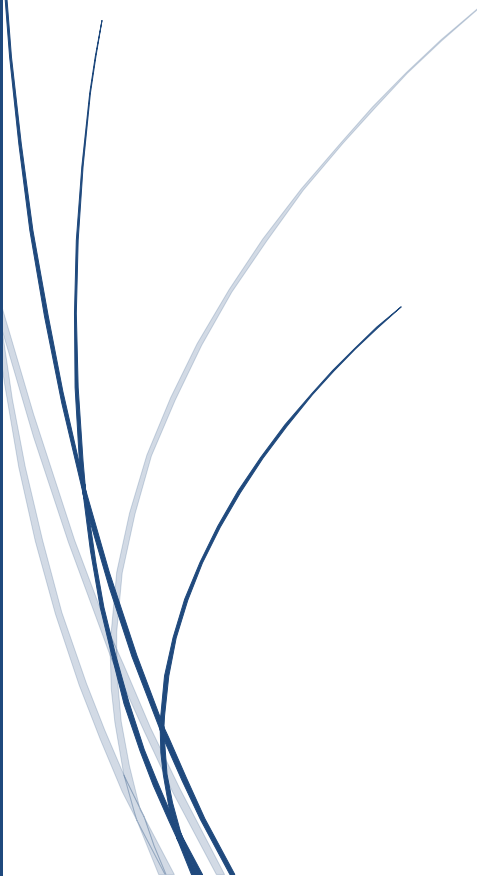




25-9-2025

Análisis del Mercado de dinero



SQL

COMISIÓN 81830

PROFESOR: LEONEL LO PRESTI

TUTORA: JENIFER GOLDFELD

ALUMNA: SILVANA TOMSIG

Sumario

- Introducción
- Objetivo
- Situación problemática
- Modelo de negocio
- Diagrama E-R
- Listado de tablas
- Scripts de creación de objetos
- Scripts de inserción de datos
- Vistas
- Funciones
- Stored Procedures
- Triggers
- Casos de prueba
- Informe analítico
- Herramientas y tecnologías utilizadas

1. Introducción

El presente documento describe el diseño e implementación de una base de datos relacional para la gestión y análisis de transacciones financieras. La solución busca optimizar el manejo de información contable y macroeconómica, permitiendo consultas ágiles, reportes consolidados y análisis multidimensional.

2. Objetivo

El proyecto tiene como objetivo principal centralizar la información financiera en un modelo relacional que permita:

- Consultas rápidas y confiables.
- Análisis multidimensional (sector, cuenta, tiempo).
- Generación de reportes útiles para la toma de decisiones.
- Integración con herramientas de analítica (Excel, Power BI, Tableau).

3. Situación Problemática

Actualmente la información financiera se encuentra en archivos dispersos, lo que genera:

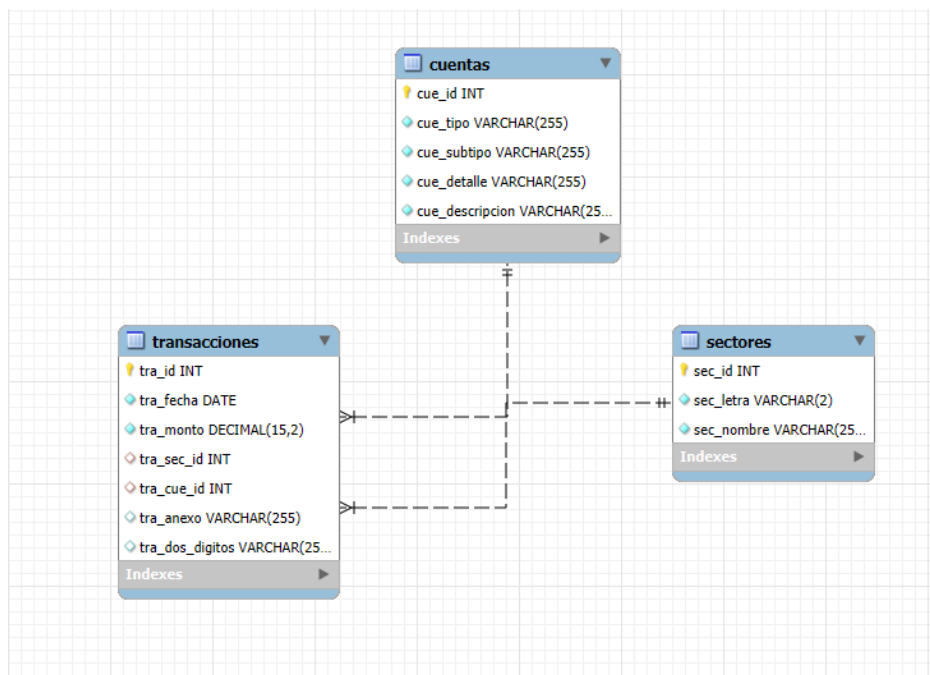
- Duplicidad de registros.
- Pérdida de integridad en los datos.
- Complejidad en consultas y reportes.
- Falta de trazabilidad.

Con la implementación de una base de datos relacional, estos problemas se solucionan, garantizando consistencia y escalabilidad.

4. Modelo de Negocio

El modelo está orientado a una organización de análisis económico-financiero, que necesita monitorear sectores económicos, gestionar transacciones y elaborar reportes contables y de gestión.

5. Diagrama E-R



6. Listado de Tablas

Tabla Sectores:

- sec_id (PK) – INT – Identificador
- sec_letra – VARCHAR(2) – Letra del sector
- sec_nombre – VARCHAR(255) – Nombre del sector

Tabla Cuentas:

- cue_id (PK) – INT – Identificador
- cue_tipo – VARCHAR(255) – Tipo de cuenta
- cue_subtipo – VARCHAR(255) – Subtipo
- cue_detalle – VARCHAR(255) – Detalle
- cue_descripcion – VARCHAR(255)

Tabla Transacciones:

- tra_id (PK) – INT – Identificador
- tra_fecha – DATE – Fecha
- tra_monto – DECIMAL(15,2) – Monto
- tra_sec_id (FK) – INT – Sector
- tra_cue_id (FK) – Cuenta
- tra_anexo – VARCHAR(255)
- tra_dos_digitos – VARCHAR(255)

7. Scripts de creación de objetos

-- Archivo: creacion_tablas.sql

-- Script para la creación de la base de datos y tablas principales

```
CREATE DATABASE IF NOT EXISTS `proyecto_final_db`;  
USE `proyecto_final_db`;
```

-- Tabla Sectores

```
CREATE TABLE IF NOT EXISTS `Sectores` (  
  `sec_id` INT AUTO_INCREMENT,  
  `sec_letra` VARCHAR(2) NOT NULL,  
  `sec_nombre` VARCHAR(255) NOT NULL,  
  PRIMARY KEY (`sec_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- Tabla Cuentas

```
CREATE TABLE IF NOT EXISTS `Cuentas` (  
  `cue_id` INT AUTO_INCREMENT,  
  `cue_tipo` VARCHAR(255) NOT NULL,  
  `cue_subtipo` VARCHAR(255) NOT NULL,  
  `cue_detalle` VARCHAR(255) NOT NULL,  
  `cue_descripcion` VARCHAR(255) NOT NULL,  
  PRIMARY KEY (`cue_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- Tabla Transacciones

```
CREATE TABLE IF NOT EXISTS `Transacciones` (  
  `tra_id` INT AUTO_INCREMENT,  
  `tra_fecha` DATE NOT NULL,  
  `tra_monto` DECIMAL(15, 2) NOT NULL,  
  `tra_sec_id` INT,  
  `tra_cue_id` INT,
```

```

`tra_anexo` VARCHAR(255),
`tra_dos_digitos` VARCHAR(255),
PRIMARY KEY (`tra_id`),
INDEX `idx_fecha` (`tra_fecha`),
FOREIGN KEY (`tra_sec_id`) REFERENCES `Sectores` (`sec_id`) ON DELETE SET NULL,
FOREIGN KEY (`tra_cue_id`) REFERENCES `Cuentas` (`cue_id`) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

8. Scripts de inserción de datos

```

-- Archivo: insercion_datos.sql
-- Inserción de datos iniciales en las tablas

```

```

USE `proyecto_final_db`;

```

```

-- Sectores

```

```

INSERT INTO `Sectores` (`sec_letra`, `sec_nombre`) VALUES
('A', 'Agricultura, ganadería y pesca'),
('B', 'Explotación de minas y canteras'),
('C', 'Industria manufacturera'),
('D', 'Energía'),
('E', 'Construcción');

```

```

-- Cuentas

```

```

INSERT INTO `Cuentas` (`cue_tipo`, `cue_subtipo`, `cue_detalle`, `cue_descripcion`) VALUES
('Cuenta Corriente', 'Bienes', 'Cobros de exportaciones', 'Exportaciones - Ingresos'),
('Cuenta Corriente', 'Servicios', 'Servicios varios', 'Servicios - Ingresos'),
('Cuenta Financiera', 'Divisas', 'Compra de divisas', 'Egresos de divisas'),
('Cuenta Corriente', 'Intereses', 'Pago de intereses', 'Intereses - Egresos');

```

```

-- Transacciones

```

```

INSERT INTO `Transacciones` (`tra_fecha`, `tra_monto`, `tra_sec_id`, `tra_cue_id`, `tra_anexo`,
`tra_dos_digitos`) VALUES
('2023-01-01', 1000000, 1, 1, '1', 'Exportación soja'),
('2023-01-05', 500000, 2, 2, '2', 'Servicios mineros'),
('2023-02-10', -200000, 3, 3, '3', 'Compra de divisas'),
('2023-03-15', -150000, 4, 4, '4', 'Pago intereses'),
('2023-03-20', 750000, 5, 1, '5', 'Obra pública');

```

9. Vistas

Las siguientes vistas fueron creadas para facilitar la consulta y análisis:

- vw_transacciones_sector
- vw_transacciones_cuenta
- vw_resumen_sector_mensual
- vw_top_sectores
- vw_balance_cuentas
- vista_promedio_monto_por_tipo_cuenta
- vista_total_mensual_anual
- vista_transacciones_maximas

-- Archivo: objetos_db.sql

-- Creación de vistas, funciones, procedimientos y triggers

USE `proyecto\_final\_db`;

-- =====

-- VISTAS

-- =====

```
CREATE OR REPLACE VIEW vw_transacciones_sector AS
SELECT t.tra_id, t.tra_fecha, t.tra_monto, s.sec_nombre, s.sec_letra
FROM Transacciones t
LEFT JOIN Sectores s ON t.tra_sec_id = s.sec_id;
```

```
CREATE OR REPLACE VIEW vw_transacciones_cuenta AS
SELECT t.tra_id, t.tra_fecha, t.tra_monto, c.cue_tipo, c.cue_subtipo
FROM Transacciones t
LEFT JOIN Cuentas c ON t.tra_cue_id = c.cue_id;
```

```
CREATE OR REPLACE VIEW vw_resumen_sector_mensual AS
SELECT YEAR(t.tra_fecha) AS anio, MONTH(t.tra_fecha) AS mes, s.sec_nombre,
SUM(t.tra_monto) AS total_mensual
FROM Transacciones t
JOIN Sectores s ON t.tra_sec_id = s.sec_id
GROUP BY anio, mes, s.sec_nombre;
```

```
CREATE OR REPLACE VIEW vw_top_sectores AS
SELECT s.sec_nombre, SUM(t.tra_monto) AS total
FROM Transacciones t
JOIN Sectores s ON t.tra_sec_id = s.sec_id
GROUP BY s.sec_nombre
ORDER BY total DESC;
```

```
CREATE OR REPLACE VIEW vw_balance_cuentas AS
SELECT c.cue_tipo, c.cue_subtipo,
       SUM(CASE WHEN t.tra_monto > 0 THEN t.tra_monto ELSE 0 END) AS ingresos,
       SUM(CASE WHEN t.tra_monto < 0 THEN t.tra_monto ELSE 0 END) AS egresos,
       SUM(t.tra_monto) AS balance_total
FROM Transacciones t
JOIN Cuentas c ON t.tra_cue_id = c.cue_id
GROUP BY c.cue_tipo, c.cue_subtipo;
```

```
CREATE OR REPLACE VIEW vista_promedio_monto_por_tipo_cuenta AS
SELECT c.cue_tipo, AVG(t.tra_monto) AS promedio_monto
FROM Transacciones t
JOIN Cuentas c ON t.tra_cue_id = c.cue_id
GROUP BY c.cue_tipo ORDER BY promedio_monto DESC;
```

```
CREATE OR REPLACE VIEW vista_total_mensual_anual AS
SELECT YEAR(tra_fecha) AS anio, MONTH(tra_fecha) AS mes, SUM(tra_monto) AS
total_mensual
FROM Transacciones
GROUP BY anio, mes ORDER BY anio, mes;
```

```
CREATE OR REPLACE VIEW vista_transacciones_maximas AS
SELECT s.sec_nombre, MAX(t.tra_monto) AS monto_maximo
FROM Transacciones t
JOIN Sectores s ON t.tra_sec_id = s.sec_id
GROUP BY s.sec_nombre ORDER BY monto_maximo DESC;
```

10. Funciones

Funciones definidas para cálculos específicos:

- fn_total_por_sector → Devuelve el total anual por sector.
- fn_crecimiento_mensual → Calcula el porcentaje de variación mensual.

Código SQL:

📄 Incluido en objetos_db.sql

11. Stored Procedures

- sp_insertar_transaccion → Inserta validando claves.
- sp_reporte_anual → Genera un consolidado anual por sector y cuenta.

☑ Incluido en objetos_db.sql

12. Triggers

Se implementó el trigger:

- trg_valida_monto → impide insertar montos nulos o cero.

☑ Incluido en objetos_db.sql

13. Casos de Prueba

Se diseñaron pruebas para validar integridad y reglas de negocio:

Caso 1: Inserción válida → la transacción se guarda.

Caso 2: Sector inexistente → error controlado.

Caso 3: Cuenta inexistente → error controlado.

Caso 4: Monto NULL o 0 → bloqueado por trigger.

Caso 5: Consulta función fn_total_por_sector → devuelve el valor esperado.

Caso 6: Consulta fn_crecimiento_mensual → devuelve % de variación.

Caso 7: Reporte sp_reporte_anual → lista totales por sector y cuenta.

14. Informe Analítico

Con las vistas generadas, se exportaron datos a Excel y Power BI para graficar:

- Ranking de sectores (vw_top_sectores).

- Evolución mensual (vista_total_mensual_anual).

☑ Insertar aquí capturas de los gráficos y un breve análisis de tendencias.

15. Herramientas y Tecnologías Utilizadas

- MySQL → motor de base de datos.

- MySQL Workbench → modelado y E-R.

- SQL → consultas y scripts.

Conclusión

La implementación de la base de datos desarrollada permitió transformar un esquema de información dispersa e inconsistente en un modelo centralizado, estructurado y escalable. El diseño de las tablas, junto con las vistas, funciones, procedimientos almacenados y triggers, asegura la integridad de los datos y facilita la generación de reportes confiables para el análisis financiero.

Asimismo, la incorporación de casos de prueba válida el correcto funcionamiento de las reglas de negocio definidas, mientras que la integración con herramientas de análisis como Excel o Power BI demuestra la capacidad del modelo para aportar valor en la toma de decisiones estratégicas.

En síntesis, el proyecto cumple con los objetivos planteados: garantiza calidad de datos, optimiza consultas y abre la posibilidad de un análisis multidimensional, posicionándose como una solución sólida y extensible para futuras necesidades de gestión y análisis económico-financiero.